AGENTIC DECOMPOSITION FOR RELIABLE LONG-HORIZON AI PLANNING IN COMPETITIVE ENVIRONMENTS

Karan Allagh[1], Raz Saremi[1], and Adam Rigby[2]
[1] *Department of Computer Science, New York University (NYU), USA*
[2] *KY, USA*
{ka3527, r.saremi}@nyu.edu, ad@mrigby.com

ABSTRACT

Large Language Models (LLMs) demonstrate strong performance on isolated tasks. Yet they remain brittle when applied to long-horizon, multi-step problems characteristic of complex socio-technical work systems, such as crowdsourced software development (CSD). We propose an agentic task-decomposition framework for CSD that operationalizes principles of distributed digital labor by breaking complex software development tasks into focused subtasks and coordinating their execution across multiple specialist agents. The framework integrates predictive modeling, reinforcement optimization, and structured chains of LLM prompting to support scalable and reliable task coordination.

Our approach combines: (1) multi-modal outcome prediction to anticipate task success and failure in advance, enabling proactive reallocation of effort within a crowdsourcing ecosystem; (2) adaptive task selection within a competitive simulation environment, where a reinforcement-trained agent dynamically prioritizes high-value tasks under resource constraints; and (3) collaborative LLM decomposition chains, in which teams of LLM agents solve subtasks with localized error correction, cross-validation, and voting mechanisms.

We train predictive models on 22,023 crowdsourced coding challenges, achieving F1 up to 0.982 for failure-mode classification using structured features and graph embeddings. In simulation, a DQN-based task-selection policy achieves a 95% win rate compared to 81% for a random baseline, while reducing uncompleted tasks by approximately fivefold. For long-form problem solving, agentic LLM workflows consistently outperform single-call baselines, completing multi-step tasks in settings where unstructured prompting fails beyond a few steps.

We further specify key hyperparameters, minimal prompting templates, and governance considerations to support reproducibility and responsible deployment. These results demonstrate how agentic decomposition—grounded in crowdsourcing and human teamwork metaphors—can enhance reliability, efficiency, transparency, and adaptability in complex AI systems.

KEYWORDS

agentic systems; task decomposition; large language models; crowdsourcing; reinforcement learning; reliability

1. INTRODUCTION

Complex real-world problems often require numerous reasoning or decision steps in sequence. Even small errors can compound over long sequences, causing standard monolithic AI agents to break down. Recent studies underline this fragility: advanced reasoning models succeed on short tasks but "illusion of thinking" experiments (e.g., extended Towers of Hanoi puzzles) find that performance collapses beyond ~8 steps (Shojaee et al., 2025). This poses a fundamental obstacle for deploying AI in domains like competitive programming, project management, or multi-stage workflows, where sustained accuracy over many dependent steps is required.

Agentic task decomposition has emerged as a promising solution (Gabriel et al., 2024; Grunde-McLaughlin et al., 2023). Instead of relying on a single agent to complete a complex task, the task is divided among many lightweight agents or subtasks, each responsible for carrying out a small part. By structuring intelligence as a team effort, this approach contains errors locally and allows for targeted bug fixes or code verifications at each step (Wu et al., 2022). Previous works have drawn analogies to

crowdsourcing: much as human workflows decompose large projects into manageable pieces, AI chains decompose reasoning into modular sub-calls (Grunde-McLaughlin et al., 2023; Bernstein et al., 2010). Notable examples include MAKER (Maximal Agentic Decomposition with k-threshold voting), which reported a million-step LLM execution with zero mistakes by orchestrating many micro-agents with voting-based error correction (Meyerson et al., 2025). Additionally, self-consistency and voting schemes can markedly increase reliability compared to naive single-shot answers (Wang et al., 2022; Meyerson et al., 2025).

Review feedback commonly emphasizes two practical gaps: (i) insufficient specification of prompting and configuration details (reproducibility), and (ii) unclear differentiation of contributions from prior agentic frameworks. We address both by providing a unified pipeline, concrete system details, and an explicit mapping from predictions to allocation to multi-agent execution.

1.1 Contributions

We make three contributions:

- A unified framework combining (a) multimodal outcome prediction, (b) reinforcement learning (RL)-based task selection, and (c) a Planner–Solver–Verifier LLM workflow for reliable long-horizon execution in competitive crowdsourced software development environments.
- Reproducibility-focused specification of agent roles, prompting strategy, and key hyperparameters for predictive and RL components.
- A cost–quality discussion plus ethics and governance considerations for deploying agentic systems in digital labor platforms and broader e-society contexts.

2. RELATED WORK

2.1 LLM Chain-of-Thought and Decomposition

There is growing interest in techniques that enable LLMs to tackle complex queries via intermediate steps. Chain-of-thought prompting improves performance on math and logic problems, but still uses a single-agent paradigm. More advanced are chained LLM workflows and Automatic Chain-of-Thought (Auto-CoT) approaches that break a problem into multiple prompt calls (Zhang et al., 2023). Wu et al. introduced AI Chains to let users compose transparent multi-step LLM processes (Wu et al., 2022), and subsequent studies extended this to self-consistency (sampling multiple reasoning paths and selecting the majority vote) (Wang et al., 2022). Such voting and consistency checks can markedly increase reliability (Wang et al., 2022; Meyerson et al., 2025). Our agentic decomposition builds on these ideas by using multiple agents in parallel for different subtasks and employing a structured, role-based aggregation of their outputs.

2.2 Hierarchical and Multi-Agent AI

Our approach is related to hierarchical and multi-level multi-agent systems. In such systems, a high-level coordinator delegates responsibilities to lower-level agents, analogous to our meta-level agent that orchestrates, monitors, and refines the behavior of specialist LLM agents. Prior work on multi-agent LLM systems has explored the construction of autonomous “societies” of agents, often focusing on emergent interaction, but frequently without structured multi-level control or mechanisms to ensure reliability and consistency. For example, Uluslu et al. used an agentic pipeline of specialized experts and a coordinator to improve robustness in native language identification (Uluslu et al., 2025). Similarly, the MAKER system achieved extreme reliability by decomposing into focused micro-agents with voting-based error correction (Meyerson et al., 2025), illustrating the power of parallelization with consensus.

2.3 Tool Use and Planner–Executor Architectures

Planner–Executor loops interleave reasoning and actions (e.g., code execution, database queries). Work on dynamic task graph generation highlights evaluation beyond accuracy, including structural quality of the decomposition (Gabriel et al., 2024). Gabriel et al. introduce metrics such as a Structural Similarity Index (SSI) and node-level F1 to assess how well an agent's task graph matches an optimal structure. Our success-versus-cost analysis complements this by treating decomposition depth as a tunable efficiency parameter and selectively choosing when to decompose. Including external tool use (e.g., interpreters, test runners, formal solvers) could further improve verification of each subtask (Gabriel et al., 2024).

2.4 Multi-Modal and Graph Representations

Predicting challenge outcomes benefits from combining text, temporal signals, and relational context. Node2Vec embeddings over the participant–challenge bipartite graph captures community structure and challenge similarities. In our experiments, adding these graph features yields F1 gains of ~0.024 to 0.072 absolute, underscoring the importance of structural context.

3. METHODOLOGY

3.1 System Overview

Figure 1 summarizes the end-to-end structure of our setting and where the proposed agentic system intervenes. A crowdsourcing platform posts challenges and collects registrations and submissions. Our agent selects a challenge, decomposes the challenge into mini tasks, executes subtasks via a multi-agent LLM workflow, and recomposes a final submission. Performance feedback can be logged for iterative refinement.

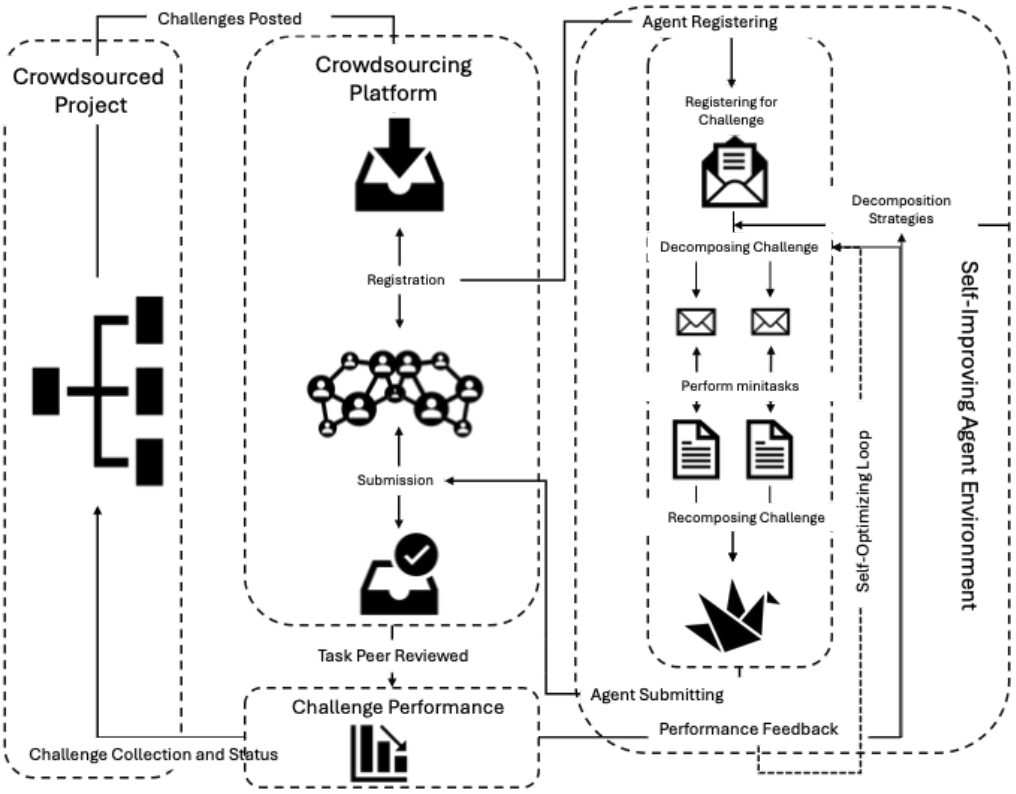

Figure 1. System overview: platform challenge lifecycle and the agentic decomposition loop used in this work.

3.2 Problem Setup and Data

Our primary experimental setting is online coding competitions. We use an export of 22,023 Topcoder challenges (Topcoder, n.d.) to test and train agent. Each challenge includes the following components:

1. *Description* Textual problem statement and requirements
2. *Metadata* Monetary Prize, Tags, Technology stack, Newcomer flags
3. *Timestamps* Registration start/end, submission deadline
4. *Outcomes* Submissions, Winner Status, Cancellation reasons

We focus on three mutually exclusive terminal outcomes:

1. *Successful:* the challenge ends successfully and at least one passing submission is awarded first place.
2. *Failed (No Winner):* the challenge runs but no winner is awarded (e.g., no submission passes review).
3. *Starved/Dropped*: the challenge is canceled early due to insufficient participation.

3.2.1 Data split

We use time-based splits to reduce temporal leakage: an 80/20 train–test split, with a validation carve-out from training where needed.

3.3 Supervised Outcome Prediction

We train supervised models to predict terminal outcomes from multimodal challenge features.

3.3.1 Feature sets

- Text features: TF-IDF and pretrained embeddings (e.g., CodeBERT-style representations) extracted from challenge descriptions.
- Time features: registration and submission durations, launch timing signals, deadline proximity indicators.
- Metadata: prize amount, technology stack, newcomer eligibility, structured fields.
- Graph embeddings: Node2Vec embeddings for each challenge node in the participant–challenge bipartite graph.

3.3.2 Models

We evaluate logistic regression (elastic net), Random Forests, and gradient boosting for classification.

3.3.3 Reproducibility details (representative settings)

- TF-IDF: 2,048 n-gram features (1–2 grams) via TfidfVectorizer.
- Node2Vec: embedding dimension d=64, walk length l=30, number of walks n=20 per node.

Table 1. Supervised prediction results (Logistic Regression) for challenge outcomes. Multimodal features include text, time, metadata, and graph embeddings.

Outcome	Text+Time F1	Multimodal F1	F1 Gain
Starved/Dropped	0.910	0.982	+0.072
Failed (No Winner)	0.917	0.952	+0.035
Successful	0.986	0.9865	+0.0005

3.4 Self-Improving Agent for Task Selection (Simulation)

We model multi-challenge management as an episodic Markov Decision Process (MDP): at each time step, the policy selects one active challenge to address (or a no-op). This stage is evaluated in a simulator as a proof-of-concept for adaptive allocation.

3.4.1 State

The state summarizes features of active challenges, including predicted outcome probabilities, time remaining to deadlines, and participation levels. To keep dimensionality manageable, we encode summary representations (e.g., top-K risky challenges).

3.4.2 Actions

The action is the index of a challenge to allocate effort to, plus a special no-op action.

3.4.3 Reward

We use a sparse terminal reward plus a small step penalty:

$$R = \sum_t (-\lambda) + \sum_i \mathbb{1}[Winner_i] . \alpha . \ prize_i$$

Where: α scales prizes and −λ encourages efficiency.

3.4.4 Policy architecture

We implement a Deep Q-Network (DQN) for discrete actions. Table 2 explains the detailed hyperparameters of this model.

Table 2. Hyperparameters for the DQN Policy Architecture.

Parameter	Value
MLP Architecture	[256, 128] fully connected layers
Activation	ReLU
Discount Factor (γ)	0.95
Optimizer	Adam
Learning Rate (η)	0.001
Replay Buffer Size	10,000 transitions
Episode Cap	30 steps

3.5 LLM Agentic Decomposition Workflow

Once the agent selects a challenge, the system deploys an LLM-based multi-agent chain. Our workflow defines three roles:

- Planner: reads the challenge description and constraints, decides on a decomposition strategy, produces subtasks.
- Solver(s): handle subtasks (often multiple Solvers in parallel for redundancy).
- Verifier: validates Solver outputs, checks constraints, selects best solution or flags fixes.

The agents communicate via intermediate prompts, and we incorporate voting and redundancy wherein multiple Solvers attempt the same subtask and the Verifier selects a consensus answer (Wang et al., 2022; Meyerson et al., 2025). By breaking down problems and introducing redundancy, the chain is more robust to individual errors and prevents propagation (Bernstein et al., 2010; Grunde-McLaughlin et al., 2023).

3.5.1 Implementation note (depth control)

In our default code path, decomposition strategies produce flat subtasks (Depth 1) to ensure strict control and to avoid runaway recursion. Benchmarks iterate strategies running one solver per strategy instance.

Table 3. Comparison of task-selection policies in the competition simulator (averaged over 100 episodes).

Agent	Avg Reward	Win Rate	Starved %
Random	69.4	81.0%	24.1%
Skill-Match	74.1	84.3%	~4.7%
Contextual Bandit	80.2	91.0%	~5.1%
DQN (ours)	108.9	95.13%	~5.14%

3.6 Adaptive Decomposition Strategy

The full multi-agent chain is powerful but resource intensive. For straightforward tasks, a single LLM call may suffice. We therefore use a Select-Then-Decompose approach (Liu et al., 2025): a meta-controller predicts whether a task is easy or hard. If easy, it uses a single-agent strategy; if hard, it triggers the full agentic workflow.

In a mixed benchmark, representative results:

- Always Single-Agent: ~72% success, lowest cost.
- Always Decompose: ~96% success, highest cost.
- Adaptive Selective: ~90% success at ~60% of full-decomposition token cost.

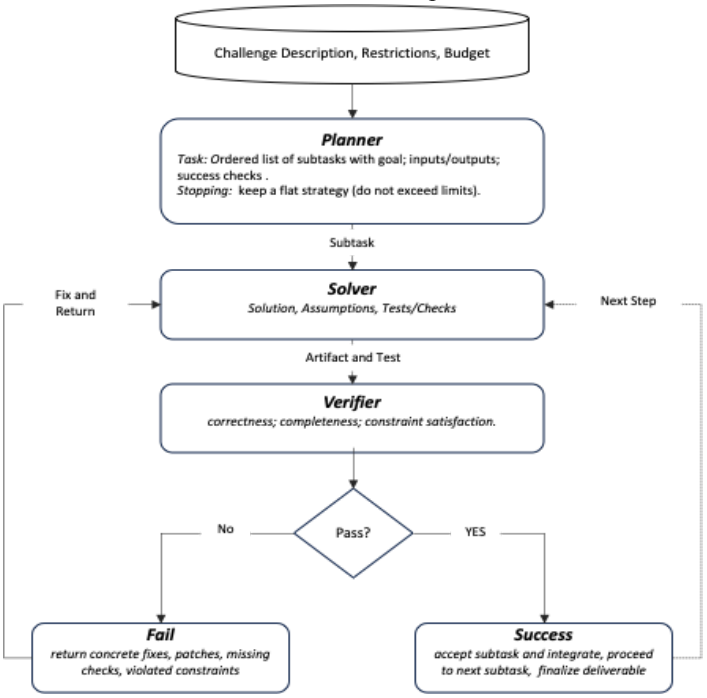

Figure 2. Planner--Solver--Verifier workflow with explicit stopping rule and verifier feedback loops.

3.7 Prompting Strategy and Minimal Templates

To improve clarity and reproducibility, we include minimal templates for each role. Figure 2 illustrates the workflow in detail.

4. EXPERIMENTAL RESULTS

4.1 Topcoder Challenge Outcomes

Using historical data as a testbed, we simulate how our system would have managed challenges. The predictor flags high-risk challenges: the selection policy decides whether to invest effort; and the LLM agents attempt to provide solutions or guidance. Because we cannot retroactively change past outcomes, we use proxy metrics and counterfactual assumptions: for example, if a challenge had zero submissions but our LLM chain would have produced a valid solution and our policy would have focused on it, we count that as a "prevented failure."

Under reasonable assumptions, the integrated system can substantially reduce starved challenges and improve overall challenge success rates, highlighting the promise of combining prediction, decision-making, and generation to actively manage complex tasks.

4.2 Long-Horizon Reasoning Tasks

We also construct algorithmic tasks to stress-test LLM decomposition. One benchmark is an expanded Tower of Hanoi puzzle with up to 20 disks, which requires $2^{20} - 1 \approx 10^{6}$ moves for an optimal solution. With ample token budget, a single-agent model fails beyond ~8 moves, consistent with Apple's long-chain reasoning study (Shojaee et al., 2025). Our multi-agent chain localizes steps into micro-tasks and uses verification and voting to reduce slip-ups. For multi-step arithmetic and logic puzzles (15+ steps), the agentic approach improves robustness relative to a single-call baseline.

4.3 Ablation Studies

We perform several ablations:

- No Graph Embeddings: Removing graph features reduces F1 by ~0.05–0.07 on failure classes.
- No Adaptive Selection: Using random selection increases starved challenges (roughly 3–5× more) and lowers aggregate reward.
- Single-Agent LLM Only: Disabling multi-agent verification collapses performance on the hardest long-horizon tasks and degrades robustness on moderate tasks.

4.4 Failure Modes

Despite improvements, failures remain:

- Poor Decomposition: If the Planner produces ambiguous or tightly coupled subtasks, Solvers may converge on the same incorrect path.
- Verifier Blind Spots: The Verifier can miss subtle criteria (e.g., edge-case bugs). External tools (unit tests, static analyzers, formal solvers) can reduce this risk (Gabriel et al., 2024).
- Budget Limits: Adaptive selection may under-decompose tasks misclassified as easy; escalation based on uncertainty signals is a natural extension (Liu et al., 2025).

5. IMPLICATIONS FOR E-SOCIETY

5.1 Relevance to E-Society Themes

Crowdsourced software development platforms are socio-technical infrastructures coordinating distributed digital labor. By improving reliability of planning and execution, agentic decomposition can support digital transformation: faster triage of work items, more transparent coordination, and decision support for allocating scarce engineering capacity. Similar patterns apply to public-sector workflow translation, civic-tech bug triage, and large-scale modernization programs.

5.2 Ethical, Safety, and Governance Considerations

Deploying agentic systems on digital labor platforms raises risks: hallucinated solutions, biased allocation, privacy leakage, and displacement concerns. Practical safeguards include: (i) audit logs of agent decisions; (ii) PII redaction; (iii) calibration checks on outcome predictors; (iv) human-in-the-loop approval for high-impact actions; and (v) explicit budget and capability limits to prevent runaway automation.

6. CONCLUSION

We proposed an agentic decomposition framework that integrates outcome prediction, adaptive decision-making, and a multi-agent LLM workflow to support reliable execution of complex tasks in competitive crowdsourced software development environments. Using 22,023 Topcoder challenges, we show that multimodal prediction with graph embeddings can identify failure risks with high F1, a learned allocation policy improves simulated win rates and reduces starvation, and multi-agent decomposition with verification improves robustness on long-horizon tasks. Future work includes incorporating richer tool augmentation, agent and worker heterogeneity, and stronger governance mechanisms for real-world deployment.

7. REFERENCES

Bernstein, M. et al., 2010. Soylent: A Word Processor with a Crowd Inside. Proceedings of UIST 2010.

Gabriel, A.G., Ahmad, A.A. and Jeyakumar, S.K., 2024. Advancing Agentic Systems: Dynamic Task Decomposition, Tool Integration and Evaluation using Novel Metrics and Dataset. arXiv:2410.22457.

Grunde-McLaughlin, M. et al., 2023. Designing LLM Chains by Adapting Techniques from Crowdsourcing Workflows. arXiv:2312.11681.

Liu, S. et al., 2025. Select-Then-Decompose: From Empirical Analysis to Adaptive Selection Strategy for Task Decomposition in Large Language Models. Proceedings of EMNLP 2025. arXiv:2510.17922.

Meyerson, E. et al., 2025. Solving a Million-Step LLM Task with Zero Errors. arXiv:2511.09030.

Shojaee, P. et al., 2025. The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity. Apple Machine Learning Research.

Topcoder, n.d. Topcoder. Available at: https://www.topcoder.com/ (Accessed: 29 January 2026).

Uluslu, A.Y. et al., 2025. Robust Native Language Identification through Agentic Decomposition. Proceedings of EMNLP 2025.

Wang, X. et al., 2022. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171.

Wu, T., Terry, M. and Cai, C.J., 2022. AI Chains: Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts. Proceedings of CHI 2022.

Zhang, Z., Zhang, A., Li, M. and Smola, A., 2023. Automatic Chain of Thought Prompting in Large Language Models. Proceedings of ICLR 2023.